WealthTracker - Flutter Wealth Tracking & Investment App

Context

Build a personal wealth tracking app that consolidates four investment categories (Gold, Stocks, USD Liquidity, Real Estate) into a single dashboard with live price tracking, historical snapshots, and charts. The user needs to track both Egyptian (EGX) and US stock markets, gold in EGP, and USD/EGP rate with minimal API usage (once/day + manual refresh).

Tech Stack

- **Framework:** Flutter (Web, macOS, iOS, Android)
- **State Management:** Signals (signals_flutter)
- **Local Storage:** Drift (reactive SQLite, with drift_dev + build_runner)
- **Charts:** fl_chart
- **HTTP:** dio
- **Auth:** flutter_screen_lock + local_auth (biometrics)
- **Other:** crypto (PIN hashing), uuid, intl, path_provider, sqlite3_flutter_libs

APIs

Service	Endpoint	Key Required	Rate Limit
GoldAPI.io	`GET https://www.goldapi.io/api/XAU/EGP`	Yes (header: `x-access-token`)	300/month
Twelve Data	`GET https://api.twelvedata.com/quote?symbol={syms}&apikey={key}`	Yes (query param)	800/day (EGX + US stocks)

Service	Endpoint	Key Required	Rate Limit
ExchangeRate	`GET https://open.er-api.com/v6/latest/USD`	No	Free/ unlimited

- EGX stock symbols use `:XCAI` suffix (e.g. `COMI:XCAI`), US stocks use plain symbols (e.g. `AAPL`, `MSFT`)
- EGX stocks are priced in EGP, US stocks are priced in USD (convert to EGP using exchange rate)
- Gold API returns `price_gram_24k`, `price_gram_22k`, `price_gram_21k`, `price_gram_18k` in EGP
- Exchange rate: `response['rates']['EGP']`

Project Structure (Feature-First Clean Architecture)

Each feature follows the clean architecture layers: **domain** (entities, repository contracts, use cases) -> **data** (repository implementations, data sources, models) -> **presentation** (screens, widgets, signals/state).

```
lib/
├── main.dart                          # DB init, dependency injection, app entry
├── app.dart                           # MaterialApp, theme, routing
│
├── core/                              # Shared across all features
│   ├── constants/
│   │   ├── app_constants.dart         # DB table names, defaults
│   │   └── api_constants.dart         # Base URLs, endpoints
│   ├── database/
│   │   ├── app_database.dart          # Drift database class (all tables)
│   │   └── app_database.g.dart        # Generated
│   ├── di/
│   │   └── service_locator.dart       # Simple DI using get_it or manual setup
│   ├── utils/
│   │   ├── currency_formatter.dart
│   │   └── date_formatter.dart
│   ├── errors/
│   │   └── app_exceptions.dart
│   └── services/                      # Shared API services
│       ├── gold_api_service.dart
│       ├── stock_api_service.dart
```

```
│       ├── exchange_rate_service.dart
│       └── price_update_service.dart  # Orchestrator
│
├── features/
│   ├── auth/
│   │   ├── domain/
│   │   │   └── auth_repository.dart       # Abstract: validatePin, setBiometric
│   │   ├── data/
│   │   │   └── auth_repository_impl.dart  # Reads/writes PIN from settings table
│   │   └── presentation/
│   │       ├── lock_screen.dart
│   │       └── auth_signals.dart          # Signal<bool> isUnlocked, pin validatio
│   │
│   ├── dashboard/
│   │   ├── domain/
│   │   │   ├── entities/
│   │   │   │   └── wealth_summary.dart    # Total wealth, per-category breakdown
│   │   │   └── wealth_repository.dart     # Abstract: getWealthSummary, getHistory
│   │   ├── data/
│   │   │   └── wealth_repository_impl.dart # Computes from all DAOs + price cache
│   │   └── presentation/
│   │       ├── dashboard_screen.dart
│   │       ├── dashboard_signals.dart      # Signal<WealthSummary>, Signal<List<Sn
│   │       └── widgets/
│   │           ├── wealth_pie_chart.dart
│   │           ├── wealth_line_chart.dart
│   │           ├── category_summary_card.dart
│   │           └── total_wealth_header.dart
│   │
│   ├── gold/
│   │   ├── domain/
│   │   │   ├── entities/
│   │   │   │   └── gold_entity.dart       # Pure domain object
│   │   │   └── gold_repository.dart       # Abstract: CRUD
│   │   ├── data/
│   │   │   └── gold_repository_impl.dart  # Drift DAO operations
│   │   └── presentation/
│   │       ├── gold_screen.dart
│   │       ├── gold_signals.dart           # Signal<List<GoldEntity>>
│   │       └── widgets/
│   │           ├── gold_item_tile.dart
│   │           └── add_gold_dialog.dart
│   │
│   ├── stocks/
│   │   ├── domain/
```

```
│   │   │   ├── entities/
│   │   │   │   └── stock_entity.dart
│   │   │   └── stock_repository.dart
│   │   ├── data/
│   │   │   └── stock_repository_impl.dart
│   │   └── presentation/
│   │       ├── stocks_screen.dart
│   │       ├── stocks_signals.dart
│   │       └── widgets/
│   │           ├── stock_item_tile.dart
│   │           └── add_stock_dialog.dart
│   │
│   ├── liquidity/
│   │   ├── domain/
│   │   │   ├── entities/
│   │   │   │   └── liquidity_entity.dart
│   │   │   └── liquidity_repository.dart
│   │   ├── data/
│   │   │   └── liquidity_repository_impl.dart
│   │   └── presentation/
│   │       ├── liquidity_screen.dart
│   │       ├── liquidity_signals.dart
│   │       └── widgets/
│   │           ├── liquidity_item_tile.dart
│   │           └── add_liquidity_dialog.dart
│   │
│   ├── real_estate/
│   │   ├── domain/
│   │   │   ├── entities/
│   │   │   │   └── real_estate_entity.dart
│   │   │   └── real_estate_repository.dart
│   │   ├── data/
│   │   │   └── real_estate_repository_impl.dart
│   │   └── presentation/
│   │       ├── real_estate_screen.dart
│   │       ├── real_estate_signals.dart
│   │       └── widgets/
│   │           ├── real_estate_item_tile.dart
│   │           └── add_real_estate_dialog.dart
│   │
│   └── settings/
│       ├── domain/
│       │   └── settings_repository.dart
│       ├── data/
│       │   └── settings_repository_impl.dart
```

```
│       └── presentation/
│           ├── settings_screen.dart
│           └── settings_signals.dart
│
└── routing/
    └── app_router.dart
```

Clean Architecture Flow (per feature)

```
Presentation (Signals + Widgets)
      | calls
      ▼
Domain (Repository contracts + Entities)
      | implemented by
      ▼
Data (Repository impl + Drift DAOs + API services)
```

- **Domain layer:** Pure Dart, no Flutter/package imports. Entities are plain classes. Repository is an abstract class.
- **Data layer:** Implements domain repositories. Depends on Drift DB and API services from core.
- **Presentation layer:** Signals hold reactive state. Screens/widgets use `Watch` widget or `.watch(context)` to rebuild on signal changes.

Data Models (Drift SQLite Tables)

GoldItems table - id (text PK), label, weightGrams (real), karat (integer: 24/22/21/18), dateAdded (dateTime) **StockItems** table - id (text PK), symbol, name, quantity (real), purchasePrice (real), market (text: 'EGX' or 'US'), currency (text: 'EGP' or 'USD'), dateAdded (dateTime) **LiquidityItems** table - id (text PK), label, amountUsd (real), dateAdded (dateTime) **RealEstateItems** table - id (text PK), projectName, purchaseDate (dateTime), purchaseAmountEgp (real), annualAppreciationPercent (real), dateAdded (dateTime) **PriceCache** table - id (integer PK auto), fetchDate (dateTime), goldPrice24k (real), goldPrice22k (real), goldPrice21k (real), goldPrice18k (real), usdToEgpRate (real) **StockPriceCache** table - symbol (text PK), price (real), fetchDate (dateTime) *(separate table for stock prices since SQLite doesn't have Map type)* **DailySnapshots** table - date (text PK 'yyyy-MM-dd'), totalValueEgp (real), totalValueUsd (real), goldValueEgp (real), stocksValueEgp (real), liquidityValueEgp (real), realEstateValueEgp (real) **AppSettings** table - key (text PK), value (text) *(key-*

value store for settings: goldApiKey, twelveDataApiKey, pinHash, useBiometric, isDarkMode)

Value Computations

- **Gold:** `weightGrams * cachedPricePerGram[karat]` (EGP)
- **Stocks (EGX):** `quantity * cachedStockPrice[symbol]` (already in EGP)
- **Stocks (US):** `quantity * cachedStockPrice[symbol] * usdToEgpRate` (USD -> EGP)
- **Liquidity:** `amountUsd * cachedUsdEgpRate` (EGP)
- **Real Estate:** `purchaseAmountEgp * pow(1 + rate/100, yearsElapsed)` (compound appreciation)
- **USD conversion:** `valueEgp / usdToEgpRate`

Price Caching Strategy

1. On app launch, query latest PriceCache row and check if `fetchDate` is today
2. If stale/empty, call all 3 APIs in parallel via `Future.wait`
3. Cache results to SQLite via Drift DAOs
4. Manual refresh button calls `forceRefresh()` (skips date check)
5. If one API fails, keep its last cached value and show warning badge

Daily Snapshots

- Taken after each successful price update
- Stored with date PK `yyyy-MM-dd` in daily_snapshots table
- Manual refresh overwrites today's snapshot via upsert
- Used by line chart for wealth-over-time visualization

Screens

1. **Lock Screen** - PIN pad + optional biometrics on launch
2. **Dashboard** - Total wealth header (EGP + USD), pie chart (4 categories), line chart (history), category summary cards, refresh button
3. **Gold** - List gold items, current gold prices header, add/edit/delete

4. **Stocks** - List stocks with gain/loss vs purchase, add/edit/delete, market selector (EGX/US) when adding
5. **Liquidity** - List USD amounts with EGP equivalent, add/edit/delete
6. **Real Estate** - List properties with compound appreciation, add/edit/delete
7. **Settings** - API keys, PIN/biometric setup, dark mode, data export/import

Build Order

Phase 1: Project Scaffold + Core

- Manually create Flutter project structure (pubspec.yaml, lib/, analysis_options.yaml, platform dirs)
- pubspec.yaml with all dependencies: drift, drift_dev, sqlite3_flutter_libs, signals_flutter, fl_chart, dio, get_it, local_auth, flutter_screen_lock, crypto, uuid, intl, path_provider
- Full folder structure (core/, features/ with domain/data/presentation per feature), constants, utils
- Service locator setup (get_it) for dependency injection

Phase 2: Core Database + Domain Entities

- All Drift table definitions in `core/database/app_database.dart`
- Domain entities for each feature (plain Dart classes, no dependencies)
- Abstract repository contracts in each feature's `domain/` layer
- main.dart with database initialization and DI registration

Phase 3: Data Layer (Repository Implementations + API Services)

- Repository implementations in each feature's `data/` layer (Drift queries)
- API services in `core/services/`: GoldApiService, StockApiService, ExchangeRateService
- PriceUpdateService orchestrator with caching logic
- Register all implementations in service locator

Phase 4: Presentation Layer (Signals)

- Signals classes per feature (e.g. `gold_signals.dart` with `Signal<List<GoldEntity>>`)
- Use `signal()` for mutable state, `computed()` for derived state

- `totalWealthSignal` (computed from all category signals + price cache)
- `snapshotHistorySignal`, `authSignal`

Phase 5: Settings + Auth Screens

- app.dart with Material 3 theme + routing
- Settings screen (API keys, PIN, dark mode)
- Lock screen (PIN + biometrics)

Phase 6: Feature Screens

- Gold, Stocks, Liquidity, Real Estate screens
- Each: list view, add dialog, edit, delete, category totals
- Widgets use `Watch` or `.watch(context)` to reactively rebuild from signals

Phase 7: Dashboard + Charts

- Pie chart (fl_chart), line chart, summary cards
- Responsive layout (mobile vs desktop)
- Wire up refresh button, snapshot triggers

Platform Notes

- **Android:** FlutterFragmentActivity for local_auth, USE_BIOMETRIC permission
- **iOS:** NSFaceIDUsageDescription in Info.plist
- **macOS:** Network entitlement (com.apple.security.network.client)
- **Web:** PIN only (no biometrics), hide biometric option. Uses `drift/web.dart` with sql.js for SQLite on web.

Verification

1. Run `flutter analyze` for static analysis
2. Run the app on web/mobile
3. Add API keys in Settings, verify prices fetch correctly
4. Add items in each category, verify values compute correctly
5. Check dashboard pie chart and line chart render
6. Test PIN lock/unlock flow
7. Test manual refresh button

8. Verify daily snapshot is recorded